# Lazy SF-FSA-VI: exact planning over a small, task-driven base

October 25, 2026

**Problem.** SF-FSA-VI [1] plans a task automaton over a set of base policies with known SFs. Entering exit $j$ in automaton state $u$ moves to $\tau(u, j)$: the accepting state $T$, another state $v$, or $u$ itself when the exit's symbol has no edge from $u$. Exit weights solve

$$w_j(u) = \begin{cases} 1 & \tau(u,j) = T \\ \gamma\, V_{\mathbf{w}(\tau(u,j))}(\varepsilon_j) & \text{otherwise,} \end{cases}$$

which Kuric et al. iterate with $V_{\mathbf{w}}(\varepsilon_j) = \max_{a,\pi} \boldsymbol{\psi}^{\pi}(\varepsilon_j, a) \cdot \mathbf{w}$ (GPI estimates at the exits). *Assumption:* the jump into the next automaton state is discounted by $\gamma$ (entering an exit takes a step), so the iteration is a $\gamma$-contraction even when an exit can be re-entered at once. The iteration is optimal if the base is a CCS. In stochastic domains the CCS is large (117–572 policies for $\epsilon$-CCSs below) and an $\epsilon$-CCS is not exact.

**Idea.** The planner only ever reads values at the weights $\mathbf{w}(u)$ it visits, and these converge. So instead of a CCS, start from the $d$ extremes and add a policy only at a visited weight where the base is provably not exact. *Certificate:* the GPI estimate $\hat{Q}_{\mathbf{w}} = \max_j Q^{\pi_j}_{\mathbf{w}}$ is a lower bound with $T\hat{Q}_{\mathbf{w}} \geq \hat{Q}_{\mathbf{w}}$, so $T\hat{Q}_{\mathbf{w}} = \hat{Q}_{\mathbf{w}}$ (one lookahead) implies $\hat{Q}_{\mathbf{w}} = Q^*_{\mathbf{w}}$. *Growth:* the metapolicy over the base first (composition), then $\pi^{\text{hyb}}$, then improvement [2].

**Algorithm 1** Lazy SF-FSA-VI

**Require:** base $L$ (the $d$ extremes, or the base left by earlier tasks), task automaton

**repeat**
  $W \leftarrow$ fixed point of Kuric's iteration over $L$ (warm-started) ▷ zero-shot; no low-level solve
  **for** every automaton state $u$ **do**
    **if** $T\hat{Q}_{\mathbf{w}(u)} = \hat{Q}_{\mathbf{w}(u)}$ **then continue** ▷ certified: GPI over $L$ is exact at $\mathbf{w}(u)$
    **end if**
    $\pi \leftarrow$ metapolicy over $L$ at $\mathbf{w}(u)$ ▷ composition of stored policies, actions $B(s)$
    **if** the Bellman gap of $\pi$ is not zero **then** $\pi \leftarrow \pi^{\text{hyb}}$, then improve until the gap is zero
    **end if**
    $L \leftarrow L \cup \{\pi\}$ (its SFs) ▷ one small refinement
  **end for**
**until** no automaton state needed a new policy
execute GPI over $L$ at $\mathbf{w}(u)$ in every $u$

**Theorem 1.** *(i) Every $W$ computed is a lower bound on the optimal weights $W^*$. (ii) Algorithm 1 terminates. (iii) At termination $W = W^*$ and the executed policy is optimal in the product MDP. (iv) In deterministic domains it adds nothing.*

*Proof.* (i) GPI estimates are lower bounds and the iteration is monotone. (ii) An uncertified $\mathbf{w}(u)$ has no optimal policy in $L$ (it would make $\hat{Q}_{\mathbf{w}(u)} = Q^*_{\mathbf{w}(u)}$), so each added policy is new; there are finitely many deterministic policies. (iii) At the stop every $\mathbf{w}(u)$ is certified, so the GPI estimates at the exits are $V^*_{\mathbf{w}(u)}$: $W = T_L W = T^* W$, whose unique fixed point is $W^*$; GPI is greedy for $Q^*_{\mathbf{w}(u)}$ in every $u$. (iv) The $d$ extremes give $\hat{Q}_{\mathbf{w}} = Q^*_{\mathbf{w}}$ for every $\mathbf{w}$ [3]. □

*Without the model*, every added policy is learned by SF Q-learning at $\mathbf{w}(u)$, started at the GPI table with the visit counts of the entries it copies (inherited counts): first on $B(s)$ (the metapolicy), then on all actions if it is not certified. The certificate itself needs the model or a simulator; below it is an oracle check that GPI is within $\delta = 10^{-3}$ of optimal at the start and the exits (*assumption*).

Table 1: With the model. Kuric et al.'s plan over the 6 extremes and over the full $\epsilon$-CCS of SFOLS (Office: $\epsilon = 2 \cdot 10^{-3}$, windy: $10^{-5}$), and the shortest OLS-order prefix of that CCS that makes it optimal (executed relative regret $\leq 10^{-9}$); Algorithm 1 from the 6 extremes on every task (fresh) or with the base carried over the task sequence (stream). Kinds: policies certified as the metapolicy / $\pi^{\text{hyb}}$ / after policy iteration. Cost in full sweeps per task (SF tables of added policies included); the baseline solves the product MDP by value iteration from scratch, without SFs. 33 Office tasks (the 3 of [1] and 30 random), 60 windy tasks.

| | Office, slip 0.1 | Office, slip 0.3 | Windy two-stage |
|---|---|---|---|
| Kuric, 6 extremes: max relative regret | $1.1 \cdot 10^{-3}$ | $3.2 \cdot 10^{-3}$ | $1.4 \cdot 10^{-2}$ |
| Kuric, full $\epsilon$-CCS: max relative regret (size) | $3.1 \cdot 10^{-6}$ (262) | $2.5 \cdot 10^{-4}$ (572) | 0 (117) |
| CCS prefix that is optimal: median (never optimal) | – (17 of 33) | 100 (5 of 33) | 11–15 (0) |
| Lazy, regret | 0 | 0 | 0 |
| fresh: rounds / policies added per task | 2.15 / 1.79 | 2.61 / 2.42 | 2.00 / 2.20 |
| stream: policies added per task / final base | 0.67 / 28 | 0.73 / 30 | 0.12 / 13 |
| kinds, fresh: metapolicy / $\pi^{\text{hyb}}$ / PI | 10 / 22 / 27 | 31 / 14 / 35 | 24 / 81 / 27 |
| kinds, stream: metapolicy / $\pi^{\text{hyb}}$ / PI | 18 / 2 / 2 | 19 / 4 / 1 | 3 / 4 / 0 |
| cost per task: lazy fresh / lazy stream | 200 / 72 | 558 / 160 | 229 / 24 |
| cost per task: product MDP from scratch | 67 | 130 | 282 |

**Where the metapolicy and [4] come in.** The metapolicy is the composition step: it decides only where stored policies disagree and keeps their common actions ("split the differences, pool the rest" [4]). $\pi^{\text{hyb}}$ adds the one action outside $B(s)$ that hedging needs [3]. The Pareto-path theorem of [4] says that neighbouring CCS policies differ by single-state flips, all optimal at their corner weight; the lazy algorithm never visits those corners, only the weights the task needs.

**Results (Table 1).**

- **Exact with a handful of policies.** Algorithm 1 is optimal on every task in 2–2.6 rounds, adding about 2 policies per task. With the base carried over, it adds 0.12–0.73 per task and ends with 13–30 policies, where Kuric's plan needs 11–15 CCS policies per windy task, a median of 100 at slip 0.3, and is never optimal on half of the slip-0.1 tasks even with all 262. In deterministic Office it adds nothing (Theorem, (iv)).
- **Refinements are compositions.** With a growing base, 91–100% of the added policies are the metapolicy or $\pi^{\text{hyb}}$ over stored policies, with no improvement step over all actions (54–80% from the extremes; in windy mostly $\pi^{\text{hyb}}$, the hedging step). Growing by GPI and policy iteration instead adds the same number of policies, but only 8–71% are the GPI policy itself.
- **Compute.** With the model the lazy plan costs about as much as solving the product MDP in Office (the metapolicy does not save sweeps over policy iteration), and 12× less in windy once the base has grown. The gain that matters is the number of low-level policies, each of which is a learning run without the model.
- **Without the model** (Office, slip 0.05, 13 tasks, extremes learned with 2.5M steps each): GPI over the learned extremes is already within $\delta$ on every task (max relative regret $4.2 \cdot 10^{-4}$), so nothing is learned beyond the 15M steps of the base, against about 130M steps for SFOLS's $\epsilon$-CCS in the same domain. WINDY$_M F$

**Limits.** The certificate needs the model or a simulator; with learned SFs it holds only up to their noise (about $10^{-3}$ here). Termination is guaranteed but the number of rounds is not bounded in terms of $d$.

# References


[1] D. Kuric, G. Infante, V. Gómez, A. Jonsson, H. van Hoof. Planning with a learned policy basis to optimally solve complex tasks. ICAPS 2024.

[2] Certify-or-Grow, `paper/article.tex`.

[3] A small SF basis for SF-FSA-VI, `notes/gpi_basis.tex`.

[4] Z. Sheebaelhamd, L. Viano, V. Cevher, C. Vernade. Split the differences, pool the rest: provably efficient multi-objective imitation. NeurIPS 2026.